

P1857R1

Modules Dependency Discovery

Published Proposal, 2019-10-07

This version:

<http://wg21.link/P1857r1>

Author:

[Michael Spencer](#) (Apple)

Audience:

EWG, SG2, SG15

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

The changes to `import` in [P1703r1] were both too much and not enough to enable fast dependency scanning via partial preprocessing. It made `import` at the start of a line anywhere in the program an *import-keyword*, breaking things like `import ->doImport ( ) ;`. Also, no similar changes were applied to `module`. This paper fine-tunes the context-sensitivity of `import` and `module`, applying the fixes discussed on the reflector.

Table of Contents

1 Effects of This Paper

2 Motivation

2.1 Fast Dependency Scanning

2.2 `import` is Too Relaxed

3 Discussion

3.1 Implementation

- 3.2 One Line Restriction
- 3.3 Extra Dependencies
- 3.4 Ship Vehicle

4 Wording

- 4.1 [lex.pptoken]
- 4.2 [basic.link]
- 4.3 [module.unit]
- 4.4 [module.global]
- 4.5 [cpp]
- 4.6 [cpp.module]
- 4.7 [cpp.glob.frag]

5 Suggested Polls

6 Acknowledgments

References

Informative References

§ 1. Effects of This Paper

At the start of phase 4 an `import` or `module` token is treated as starting a directive and are converted to their respective keywords iff:

- After skipping horizontal whitespace are
 - at the start of a logical line, or
 - preceded by an `export` at the start of the logical line.
- Are followed by an identifier pp token (before macro expansion), or
 - `<`, `"`, or `:` (but not `::`) pp tokens for `import`, or
 - `:` for `module`

Otherwise the token is treated as an identifier.

Additionally:

- The entire `import` or `module` directive (including the closing `;`) must be on a single logical line and for `module` must not come from an `#include`.
- The expansion of macros must not result in an `import` or `module` directive introducer that was not there prior to macro expansion.
- A `module` directive may only appear as the first preprocessing tokens in a file (excluding the global module fragment.)

Failure to meet these additional requirements makes the program ill-formed (rather than not being interpreted as a directive).

Status Quo	This Paper
<pre>export module x ; // ✓</pre>	<pre>export module x ; // ✗</pre>
<pre>// -Dm="export module x;" m // ✓</pre>	<pre>// -Dm="export module x;" m // ✗</pre>
<pre>module; #define m x export module m; // ✓</pre>	<pre>module; #define m x export module m; // ✓</pre>
<pre>module; #if F00 export module foo; // ✓ #else export module bar; // ✓ #endif</pre>	<pre>module; #if F00 export module foo; // ✓ #else export module bar; // ✓ #endif</pre>
<pre>module; #define EMPTY EMPTY export module m; // ✓</pre>	<pre>module; #define EMPTY EMPTY export module m; // ✗</pre>
<pre>#if MODULES module; export module m; // ✗ #endif</pre>	<pre>#if MODULES module; export module m; // ✗ #endif</pre>

Status Quo	This Paper
<pre> #if MODULES export module m; // ✓ #endif </pre>	<pre> #if MODULES export module m; // ✗ #endif </pre>
<pre> module y = {}; // ✗ ::import x = {}; // ✓ ::module y = {}; // ✓ import::inner xi = {}; // ✗ module::inner yi = {}; // ✓ namespace N { module a; // ✓ import b; // ✗ } #define MAYBE_IMPORT(x) x MAYBE_IMPORT(import <a>; // ☠) #define EAT(x) EAT(import <a>; // ☠) void f(Import *import) { import->doImport(); // ✗ } </pre>	<pre> module y = {}; // ✖ ::import x = {}; // ✓ ::module y = {}; // ✓ import::inner xi = {}; // ✓ module::inner yi = {}; // ✓ namespace N { module a; // ✖ import b; // ✖ } #define MAYBE_IMPORT(x) x MAYBE_IMPORT(import <a>; // ☠) #define EAT(x) EAT(import <a>; // ☠) void f(Import *import) { import->doImport(); // ✓ } </pre>

Legend:

- ✓ The declaration was accepted, but may not be a modules declaration.
- ✖ A token was identified as a modules keyword, but the program is later found to be ill-formed.
- ✗ Ill-formed.
- ☠ Undefined behavior

§ 2. Motivation

§ 2.1. Fast Dependency Scanning

Fast dependency scanning relies on partial preprocessing. Several people are working on fast dependency scanning for modules, and they all share the common trait of skipping over non-directives. [clang-scan-deps](#) is one such tool and is a representative motivating usecase for these changes.

clang-scan-deps works by minimizing source code via partial preprocessing. Partial preprocessing works by running phases 1-3 of translation. Minimization proceeds by throwing away every line which is not a preprocessing directive, then removing all preprocessing directives that can't impact the set of dependencies for that file. Up to C++17 this means it only needs to keep `#define`, `#undef`, `#if*`, `#else`, `#elif`, `#endif`, and `#include`. And only the bodies of `#ifs` that contain a preprocessing directive that could impact dependencies. This reduces most files down to their header guards and list of `#includes`, and is correct for standard C++ except for abuse of `__LINE__` such as `#if __LINE__ > 456`. The only other things that break it are compiler extensions which do not follow the parsing rules of `#` directives such as `_Pragma("push_macro(\"X\")")`, or `.incbin` in inline assembly. Additionally, abuse of the compiler extension `__COUNTER__` can break this.

This minimization is context free, and is done *once* for each file in the entire build.

Then the minimized source file is fully preprocessed by running phases 1-4, with `#includes` resolving to their minimized equivalent, macros expanded, and all preprocessing directives executed.

Minimization followed by using clang's full preprocessor currently provides about a 9x speed up (~3 seconds vs ~28 seconds out of a 15m build on a 18 core iMac Pro) over no minimization at scanning all of llvm and clang (~7k files and ~3.8m LoC). We do not have access to a large C++20 modules only (no header units) codebase to do tests on, but that case still requires scanning the entire file due to the potential for `#include` based [x-macros](#) which will not be going away with modules, as they are intentionally non-modular includes. There will still be a difference in that we no longer have normal includes, and thus don't need to preprocess a header for each TU it's used in.

Of the 3s only 33ms are spent minimizing, while the majority of the time is spent in the full lexer, preprocessor, and header search. We believe we can reduce this overhead. This overhead is important as it's on the critical path. Any speedup here turns into a direct reduction in build latency, which is important for extremely parallel builds.

C++20 currently breaks this approach. [\[P1703r1\]](#) resolved the issue for `import`, but dependency scanning also needs to find `module` declarations, and the current rules are not enough.

§ 2.2. `import` is Too Relaxed

[P1703r1] went too far in fixing the dependency scanning issue with `import`. Now any line starting with `import` is treated as an import directive, even if it obviously couldn't be. This breaks a decent amount of real code with function arguments or local variables named `import`, as any use of them without a prefix is treated as an import directive, and prepending `::` doesn't work, you must wrap some part of the expression in `()`.

§ 3. Discussion

§ 3.1. Implementation

There were some concerns expressed during the discussion of [P1703r1] that there may be performance or complexity issues. I have implemented this fix in Clang. It was rather simple to implement, with the most complex part being doing token lookahead while lexing (which isn't that complicated). Measurement did not show any performance impact, which I expect to be due to the codepath only occurring with the `import` and `module` tokens.

§ 3.2. One Line Restriction

To be fully resilient, a module's directive must be entirely on a single logical line. You could have a rule that the `;` must not come from a macro and say that the directive extends to the next `;`, but that is incorrect:

```
#define f(x) "blah" #x "blah"
import f(
    ;
);
```

§ 3.3. Extra Dependencies

```
#define eat(x)
eat(
    import <a>;
);
```

In the WD this has undefined behavior due to [\[cpp\]/2](#)

A preprocessing directive consists of a sequence of preprocessing tokens that satisfies the following constraints: The first token in the sequence, referred to as a directive-introducing token, is a `#` preprocessing token, an import preprocessing token, or an export preprocessing token immediately followed by an import preprocessing token, that (at the start of translation phase 4) either begins with the first character in the source file (optionally after white space containing no new-line characters) or follows white space containing at least one new-line character.

and [\[cpp.replace\]/11](#)

If there are sequences of preprocessing tokens within the list of `[macro]` arguments that would otherwise act as preprocessing directives, the behavior is undefined.

A minimizer will transform this into just `import <a>;` which when preprocessed will form a valid module import. There are two outcomes of this:

- The dependency scanner emits an error because `<a>` is not a header unit or it is able to detect the usage in a macro.
- The dependency scanner succeeds and then the compiler sees the code and ~~ignores imports/errors~~ due to undefined behavior.

This is not the best outcome, as an implementation is not guaranteed to emit a diagnostic, from my tests of clang, msvc, gcc, and icc, only clang rejects `#include` in this case, and no compiler rejects an import directive (I plan to fix this in clang as part of implementing P1703 and this paper). This should be fixed by changing [\[cpp.replace\]/11](#) to be ill-formed instead of UB, but that deserves a separate paper. With that change though the possible outcomes would be:

- The dependency scanner emits an error because `<a>` is not a header unit or it is able to detect the usage in a macro.
- The dependency scanner succeeds and then the compiler sees the code and rejects it because directives cannot appear as a macro argument.

Both of these outcomes are fine as their result is the same. If a partial preprocessing would ever get the dependencies wrong, the compiler will reject the program as ill-formed.

§ 3.4. Ship Vehicle

This needs to ship with modules as this is a backwards incompatible change. Thus it needs to be in C++20.

§ 4. Wording

Wording Note

This is a wording note. Its purpose is to help clarify the intent of the wording in this document to the members of the committee. It is not an instruction to the editor.

§ 4.1. [lex.pptoken]

```
preprocessing-token:  
    header-name  
    import-keyword  
    module-keyword  
    ...
```

4 The *import-keyword* is produced by processing an import directive ([cpp.import]) , and the *module-keyword* is produced by preprocessing a module directive ([cpp.module]). ~~and has no~~
[Note: Neither has any associated grammar productions. —end note]

§ 4.2. [basic.link]

```
translation-unit:
    top-level-declaration-seqopt
    global-module-fragmentopt module-declaration top-level-declaration-...
private-module-fragment:
    module module-keyword : private ; top-level-declaration-seqopt
```

...

3 A token sequence beginning with **export**_{opt} **module** and not immediately followed by **::** is never interpreted as the *declaration* of a *top-level-declaration*.

§ 4.3. [module.unit]

```
module-declaration:
    exportopt module module-keyword module-name module-partitionopt att...
```

§ 4.4. [module.global]

```
global-module-fragment:
    module module-keyword ; top-level-declaration-seqopt
```

§ 4.5. [cpp]

1 A *preprocessing directive* consists of a sequence of preprocessing tokens that satisfies the following constraints: The first token^s in the sequence, referred to as a *directive-introducing token subsequence*, is a # preprocessing token, an **import** or **module** preprocessing token, or an **export** preprocessing token immediately followed by an **import** or **module** preprocessing token, that is followed by a *header-name*, *identifier*, or **:** preprocessing tokens for **import**, or is followed by an *identifier* or **:** preprocessing tokens for **module**, and that (at the start of translation phase 4) either begins with the first character in the source file (optionally after white space containing no new-line characters) or follows white space containing at least one new-line character. The last token in the sequence is the first new-line character that follows the first token in the sequence. A new-line character ends the preprocessing directive even if it occurs within what would otherwise be an invocation of a function-like macro. *[Example:*

```
// These are examples of directive-introducing token subsequences
#
module ;
export module leftpad
import <string>
export import "squee"
import rightpad
import :
// These are not directive-introducing token subsequences
module
.;
import ::
import ->
—end example]
```

preprocessing-file:

...

control-line:

```
# include pp-tokens new-line
exportopt import header-name pp-tokens new-line
exportopt import identifier pp-tokens new-line
exportopt import : pp-tokens new-line
exportopt module identifier pp-tokens new-line
exportopt module ; new-line
```

...

Wording Note

This is intended to match any logical lines that start with:

- #
- `exportopt module ;`
- `exportopt module identifier`
- `exportopt import :`
- `exportopt import <`
- `exportopt import "`
- `exportopt import identifier`

And that the entirety of the above is a *directive-introducing token subsequence*.

This should only require examining up to two characters after skipping horizontal whitespace.

Wording Note

There's a potential issue here with:

```
export module m;  
#define export static  
using import = int;  
export import(*a);
```

If this is allowed, the lexer needs to do two token lookahead when it sees an `export` due to [\[cpp\]/6](#) to tell if this is a directive or not.

During discussion at the SG15 meeting after CppCon Gaby and Boris said that this lookahead/backtracking was fine for MSVC and GCC respectively. I have implemented this in Clang.

2 A text line shall not begin with a `#preprocessing token` *directive-introducing token subsequence*. A *conditionally-supported-directive* shall not begin with any of the directive names appearing in the syntax. A *conditionally-supported-directive* is conditionally-supported with implementation-defined semantics.

§ 4.6. [cpp.module]

Add a new section.

pp-module:

export_{opt} **module** *pp-tokens*_{opt}; *new-line*

1 No part of *pp-module* shall be produced directly or indirectly via source file inclusion ([cpp.include]).

Wording Note

This replaces the part in [cpp.glob.frag] and makes it apply even without a global module fragment.

Wording Note

Do we need "directly or indirectly"? It was copied from the wording in [cpp.glob.frag]/1

2 At the start of phase 4 of translation a *pp-module* directive shall appear only as the first preprocessing tokens in the translation unit or as the second *pp-module* in the *pp-global-module-fragment*.

3 Any preprocessing tokens after the **module** preprocessing token in the **module** *control-line* are processed just as in normal text. [Note: Each identifier currently defined as a macro name is replaced by its replacement list of preprocessing tokens. —end note]

4 The **module** preprocessing token is replaced by the *module-keyword* preprocessing token. [Note: This makes the line no longer a directive so it is not removed at the end of phase 4. —end note]

§ 4.7. [cpp.glob.frag]

pp-global-module-fragment:

~~module ; pp-balanced-token-seq module~~

pp-module pp-balanced-token-seq new-line pp-module

Wording Note

I'm not sure this *new-line* is needed, but nothing in the grammar for *pp-module* requires it otherwise.

1 If, ~~at the first two preprocessing tokens at the~~ start of phase 4 of translation ~~are~~ *module ;*, ~~the~~ source file begins with a *pp-module* of the form *module ;*, the result of preprocessing shall begin with a *pp-global-module-fragment* for which all preprocessing-tokens in the *pp-balanced-token-seq* were produced directly or indirectly by source file inclusion ([cpp.include]) ~~and for which the~~ second module preprocessing-token was not produced by source file inclusion or macro replacement (15.5). Otherwise, the first two preprocessing tokens at the end of phase 4 of translation shall not be *module ;*.

§ 5. Suggested Polls

- C++ dependencies should be discoverable without full preprocessing.
- Forward P1857R1 to CWG for inclusion in C++20 as a resolution to NB comments <comments>.

§ 6. Acknowledgments

Thanks to Boris Kolpackov for writing P1703, Mathias Stearn and Walter Brown for feedback on this paper and to Richard Smith, Corentin, Gabriel Dos Reis, Ben Craig, Matthew Woehlke, and Nathan Sidwell for discussion regarding this issue.

§ References

§ Informative References

[P1703r1]

Boris Kolpackov. [Recognizing Header Unit Imports Requires Full Preprocessing](https://wg21.link/p1703r1). 19 July 2019.

URL: <https://wg21.link/p1703r1>